DUKE COMPUTE CLUSTER TUTORIAL

Jia Lu

You Lab, Duke University

September 2020

* Please do not share outside the lab



CONTENT

- About Duke Compute Cluster
- Do you really need the cluster?
- General workflow
- Other things you should know about the cluster
- Additional resources

WHAT IS DUKE CLUSTER?

The Duke Compute Cluster (DCC, https://rc.duke.edu/dcc/) is a general purpose high performance/high-throughput installation, and it is fitted with software used for a broad array of scientific projects. It consists of machines that the University has provided for community use and that researchers have purchased to conduct their research.

Duke
UNIVERSITY
RESEARCH
COMPUTING

DO YOU REALLY NEED THE CLUSTER FOR YOUR RESEARCH?

You need the cluster if:

- Your job requires a lot of computational resources (ex. too slow to run on a personal computer)
- You need to run many jobs in parallel

If your job doesn't meet these criteria, it may actually be slower to use the cluster because of the complex setup

Limitations of DCC:

- If your job demands very high computational power, consider other resources (AWS, GCP, XSEDE and etc)
- The cluster can be slow due to overload and low user priority

BEFORE START

- Email me (jia.lu@duke.edu) your NetID, so I can create a Duke Cluster account for you
- If you are new to research computing, go through this tutorial to learn about the basic usage of the cluster
- If you use the cluster from off-campus, use Duke VPN (https://oit.duke.edu/net-security/network/remote/vpn/) to avoid two-factor authentication every time

Prerequisites:

Comfort with the Linux command line

GENERAL WORKFLOW OF USING THE CLUSTER

0. Get the code and files that you need ready

1. Upload your code and files to the cluster

2. Log into the cluster

3. Submit your job

4. Check or change your job status

5. When the job is done, download your results

STEP 0. GET THE CODE AND FILES THAT YOU NEED READY

You need:

1. A SLURM script for computational job scheduling
2. The code you want to run (MATLAB, Python, R, C++ and etc.)

SLURM (https://slurm.schedmd.com/documentation.html) is an open source, fault-tolerant, and highly scalable cluster management and job scheduling system for large and small Linux clusters. It has three key functions. First, it allocates exclusive and/or non-exclusive access to resources (compute nodes) to users for some duration of time so they can perform work. Second, it provides a framework for starting, executing, and monitoring work (normally a parallel job) on the set of allocated nodes. Finally, it arbitrates contention for resources by managing a queue of pending work.

STEP 0.1. WRITE A SLURM SCRIPT

Save the following script as .q or .sh file

Long-form

```
#!/bin/bash
#SBATCH --output=slurm.out # Give the output file name
#SBATCH --error=slurm.err # Give the error file name
#SBATCH --mem=100 # Set the required memory per node as 100MB
/opt/apps/MATLAB/R2012b/bin/matlab -nojvm -nodisplay -r MATLAB_code # Run
MATLAB_code with R20212b MATLAB
```

Short-form

```
#!/bin/bash
#SBATCH -o slurm.out # Give the output file name
#SBATCH -e slurm.err # Give the error file name
#SBATCH --mem=100 # Set the required memory per node as 100MB
/opt/apps/MATLAB/R2012b/bin/matlab -nojvm -nodisplay -r MATLAB_code # Run
MATLAB_code with R20212b MATLAB
```

STEP 0.1. WRITE A SLURM SCRIPT

MATLAB

```
#!/bin/bash
#SBATCH -o slurm.out
#SBATCH --mem=100
/opt/apps/MATLAB/R2012b/bin/matlab -nojvm -nodisplay -r MATLAB_code # No
extension for MATLAB code
```

Python

```
#!/bin/bash
#SBATCH -o slurm.out
#SBATCH --mem=100
module load Python-GPU/3.6.5
python python_code.py
```

R

```
#!/bin/bash
#SBATCH -o slurm.out
#SBATCH --mem=100
module load R/3.6.0
R CMD BATCH R_code.R
```

Check the directory and version of the applications on the cluster

STEP 0.1. WRITE A SLURM SCRIPT

Job array

```
#!/bin/bash
#SBATCH --output=slurm.out
#SBATCH --array=1-100 # Run MATLAB_code 100 times in parallel
#SBATCH --mem=100
/opt/apps/MATLAB/R2012b/bin/matlab -nojvm-nodisplay–singleCompThread-r
“rank=$SLURM_ARRAY_TASK_ID;MATLAB_code;quit”
```

Change the flags according to your job! These are just examples!

STEP 0.2. ADAPT YOUR CODE FOR SLURM

For job arrays, add the following lines to the beginning of your code, so that your code can access a unique task ID during each job

MATLAB

```
taskID = str2num(getenv('SLURM_ARRAY_TASK_ID'));
```

Python

```
import os
taskID=int(os.environ['SLURM_ARRAY_TASK_ID'])
```

R

```
taskID <- as.integer(Sys.getenv(SLURM_ARRAY_TASK_ID))
```

STEP 0.2. ADAPT YOUR CODE FOR SLURM

You may use the task ID to run the same code with different parameters

Python

```
parameter_list = [1,2,3]
parameter = parameter_list[taskID-1]
```

Or save your results as different files

Python

```
filename = path + str(taskID) + '.txt'
with open(filename,'w') as f:
   f.write(my_results)
```

STEP 1. UPLOAD YOUR CODE AND FILES

Go to terminal, use the command below to upload your files

* You may need to log in and create the destination folder first (see step 2)

```
scp [local_location_of_file] [NetID]@dcc-slogin.oit.duke.edu:
[cluster_destination_of_file]
```

You will be asked to enter your password, it’s the password you use to log into Dukehub

STEP 2. LOG INTO THE CLUSTER

SSH to your account

```
ssh [NetID]@dcc-slogin.oit.duke.edu
```

STEP 3. SUBMIT YOUR JOB

Go to the directory containing your code, submit a job

```
sbatch [slurm_script_file]
```

STEP 4. CHECK OR CHANGE YOUR JOB STATUS

View the status of the running jobs under your account

```
squeue -u [NetID]
```

Cancel a job

```
scancel [job_ID]
```

STEP 5. DOWNLOAD YOUR RESULTS

1. Log out of your cluster account

```
exit
```

2. Download your results

```
scp [NetID]@dcc-slogin.oit.duke.edu:[cluster_location_of_file]
[local_destination_of_file]
```

Voila!

OTHER THINGS YOU SHOULD KNOW ABOUT THE CLUSTER

DCC partition and user priority

`common`	DCC core nodes (up to 64 GB RAM, the default partition if partition is not specified)
`scavenger`	Use nodes owned by other labs in “low priority” (kill and requeue preemption)
`gpu-common`	DCC GPU nodes
`scavenger-gpu`	Use GPU nodes owned by other labs in “low priority” (kill and requeue preemption)

Prioritize using the “common” nodes, the You lab does not own any nodes

You can specify the partition in the SLURM script

```
#SBATCH -p gpu-common --gres=gpu:1 # Use gpu-common, 1 GPU per node
#SBATCH -c 4 # 4 CPUs per node
```

View all partitions and nodes on the cluster

```
sinfo
```

OTHER THINGS YOU SHOULD KNOW ABOUT THE CLUSTER

Storage space

10 GB in your home directory

```
/hpc/home/[NetID]
```

1 TB in your folder under the You lab group

```
/hpc/group/youlab/[NetID]
```

Check installed applications on the cluster

```
module avail
```

If an application is not installed, you can email DCC and ask them to install for you

TO LEARN MORE

- DCC user guide (https://rc.duke.edu/dcc/dcc-user-guide/)
- DCC workshops (https://rc.duke.edu/training/)
- DCC office hours(https://rc.duke.edu/zoom-in-office-hours/)
- HARDAC, another computing resource provided by Duke (https://genome.duke.edu/cores-and-services/computational-solutions/compute-environments-genomics)